

Name of Experiment: Write a program to implement Producer Consumer Problem

Problem Statement: Write a program in C to implement producer Consumer Problem

Software /Hardware Requirements: GCC Compiler, Personal Computer

Theory:

The Producer-Consumer problem is a classic synchronization issue in operating systems. It involves two types of processes: producers, which generate data, and consumers, which process that data. Both share a common buffer. The challenge is to ensure that the producer doesn't add data to a full buffer and the consumer doesn't remove data from an empty buffer while avoiding conflicts when accessing the buffer. In this article, we are going to solve this problem by using semaphores.

What is a Semaphore?

A semaphore is a synchronization tool used in computing to manage access to shared resources. It works like a signal that allows multiple processes or threads to coordinate their actions. Semaphores use counters to keep track of how many resources are available, ensuring that no two processes can use the same resource at the same time, thus preventing conflicts and ensuring orderly execution.

A **semaphore** *S* is an integer variable that can be accessed only through two standard operations: `wait()` and `signal()`. The `wait()` operation reduces the value of the semaphore by 1 and the `signal()` operation increases its value by 1.

Producer–Consumer Problem Algorithm

Step 1: Start.

Step 2: Initialize the buffer size.

Step 3: Initialize semaphores:

- `empty = BUFFER_SIZE`
- `full = 0`
- Initialize mutex.

Step 4: Producer

1. Wait for an empty space using `wait(empty)`.
2. Lock the buffer using mutex.
3. Produce an item.
4. Insert the item into the buffer.
5. Unlock the buffer.
6. Signal full.
7. Repeat until all items are produced.

Step 5: Consumer

1. Wait for a filled space using wait(full).
2. Lock the buffer using mutex.
3. Remove an item from the buffer.
4. Consume the item.
5. Unlock the buffer.
6. Signal empty.
7. Repeat until all items are consumed.

Step 6: Destroy semaphores and mutex.

Step 7: Stop.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;
```

```

for (int i = 1; i <= 5; i++)
{
    item = i;

    sem_wait(&empty);
    pthread_mutex_lock(&mutex);

    buffer[in] = item;
    printf("Producer produced: %d\n", item);
    in = (in + 1) % SIZE;

    pthread_mutex_unlock(&mutex);
    sem_post(&full);
}

return NULL;
}

void *consumer(void *arg)
{
    int item;

    for (int i = 1; i <= 5; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % SIZE;
    }
}

```

```

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Producer produced: 1

Producer produced: 2

Producer produced: 3

Producer produced: 4

Producer produced: 5

Consumer consumed: 1

Consumer consumed: 2

Consumer consumed: 3

Consumer consumed: 4

Consumer consumed: 5